

Final Project Report



Bank Account Fraud Detection

Berniss Chow - A0237829N
Joan Wong Ya Sian - A0238362A
Ramasamy Shreya - A0244253L
Parthasarathy Sreenithi - A0238749L

DSA4263 Sense-making Case Analysis: Business and Commerce

17 April 2025

1. Abstract.....	2
2. Introduction.....	2
3. Data.....	3
3.1 Dataset Composition.....	3
3.2 Significant Features.....	3
3.3 Target Label.....	4
3.4 Data Collection and Validation.....	5
4. Methods.....	5
4.1 Data Cleaning and Pre-Processing.....	5
4.2 Feature Engineering.....	6
4.3 Encoding.....	8
4.4 Resampling Methods.....	9
4.5 Test/Train Split, Holdout and Validation Approach.....	9
4.6 Exploratory Data Analysis Insights.....	10
4.6.1 Hypothesis 1: Missing address history is indicative of higher fraud risk.....	10
4.6.2 Hypothesis 2: Abnormally high short-term velocity suggests automated or coordinated fraud behaviour.....	11
4.6.3 Hypothesis 3: Higher income applicants are more likely to be fraudulent.....	12
4.7 Baseline Model.....	13
4.8 Challenger Models.....	14
4.8.1 Gradient Boosting.....	16
4.8.2 Random Forest.....	17
4.8.3 Model Ensemble: Blended Bagging and Boosting.....	20
5. Results.....	22
6. Conclusion.....	23
6.1 Hypothetical Pipeline Implementation.....	23
6.2 Recommendations for Detecting Fraud Patterns.....	24
7. Appendix.....	25
7.1 Data Dictionary.....	25
7.2 Citations.....	27

1. Abstract

The Bank Account Fraud (BAF) dataset is a large-scale, synthetic dataset built to detect first-party fraud in bank account applications. It includes 1 million records with behavioural, demographic, and device-related features, posing a rare-event classification challenge with strong temporal structure. Our analysis was guided by three fraud hypotheses: missing address history indicates risk, high short-term application velocity signals automation, and higher-income applicants may be more fraudulent. We engineered features accordingly, applied Synthetic Minority Over-sampling Technique (SMOTE) to address class imbalance, and tested several models including XGBoost and Random Forest. The final ensemble model significantly outperformed others with near-perfect precision and recall. These results underscore its strong fraud detection capability, especially in identifying subtle behavioural signals. We recommend deploying the ensemble model, which balances accuracy and interpretability, aided by LIME explanations to ensure transparency in predictions which is an essential requirement for real-world fraud prevention systems.

2. Introduction

The dataset selected for this project is the BAF dataset, a large-scale, synthetic yet realistic dataset specifically designed for evaluating fraud detection systems in bank account applications. It contains 1 million labelled records generated using a Conditional Tabular Generative Adversarial Network (CTGAN) trained on anonymised real-world data. Each record includes a variety of feature-engineered attributes representing applicant behaviour, contact validity, financial status, and device signals, along with a binary target label indicating whether the application was fraudulent.

Specifically, this dataset focuses on first-party application fraud, where individuals attempt to open new accounts using falsified or manipulated information. It does not cover transactional fraud, account takeovers, or other post-application abuse.

A key aspect of this dataset is its temporal structure, made possible through the inclusion of a “month” feature. This allows for realistic training and testing setups where models are evaluated on future data, a crucial factor in operational fraud detection systems. Moreover, the dataset includes three protected attributes (age group, employment status, and income percentile), enabling fairness-aware model development and bias assessment.

The scope of this problem extends beyond banking. Industries such as e-commerce, telecommunications, and insurance also face similar risks from fraudulent applications, which can lead to identity theft, regulatory penalties, and substantial financial losses. Crucially, machine learning models trained on this dataset not only detect fraud but can also identify root causes, such as unusual device fingerprints or inconsistent address histories. These insights empower businesses to implement preemptive safeguards, thereby preventing fraud before it occurs and reducing operational overhead.

According to LexisNexis (2022), U.S. financial institutions spend an average of \$4.23 in remediation costs for every \$1 lost to fraud. Thus, even small improvements in fraud detection accuracy can lead to significant cost savings and operational efficiency. Compared to existing literature, where fraud detection research often focuses on post-account

behaviours or uses imbalanced, real-world datasets with sparse labels, the BAF dataset offers a controlled yet realistic environment for testing fraud detection pipelines at the application stage. For instance, PaySim (Lopez-Rojas et al., 2016) was developed to simulate mobile money fraud scenarios due to the lack of accessible transaction data, providing a precedent for using synthetic datasets in fraud detection. While PaySim focuses on mobile money transfers, the BAF dataset expands this approach by targeting first-party fraud in bank account applications, thus extending the scope of fraud research into earlier stages of financial service access.

As such, solving the fraud detection challenge not only enhances operational efficiency but also strengthens trust and resilience in digital financial ecosystems.

3. Data

3.1 Dataset Composition

The dataset is designed to mimic realistic fraud patterns and behaviours while ensuring data privacy. It serves as a benchmark for evaluating the effectiveness and fairness of fraud detection models in financial services. It contains 1,000,000 rows and 32 columns, where each row represents an individual bank account application. Among the 32 columns, 31 are features describing the applicant's demographics, behavioural traits, and device-related metadata, while the remaining column, *fraud_bool*, serves as the binary target label indicating whether the application was fraudulent (1) or legitimate (0).

3.2 Significant Features

A number of significant features in this dataset provide rich behavioural context that is essential for modelling fraud.

Table 1: Description of Significant Features

	Feature	Description
1	<i>fraud_bool (Target)</i>	Binary variable indicating if the application is fraudulent (1 = Fraud, 0 = Legitimate).
2	<i>velocity_6h, velocity_24h, velocity_4w</i>	Number of applications submitted from the same device in the last 6 hours, 24 hours, and 4 weeks. Fraudulent users often submit multiple applications in bursts.
3	<i>device_fraud_count</i>	Number of previously detected fraudulent applications from the same device. A strong fraud indicator.
4	<i>email_is_free</i>	Binary variable (0 = Paid email, 1 = Free email). Fraudsters often use free email providers due to the ease of creation.
5	<i>session_length_in_minutes</i>	Time spent on the bank's website during the application. Suspiciously short or long sessions may indicate automated fraud attempts.

The dataset includes both categorical, continuous numerical and binary features. In some features, missing values are encoded with out-of-range placeholders such as -1 or negative values.

3.3 Target Label

The target label, *fraud_bool*, defines the classification task: predicting whether an account application is fraudulent.

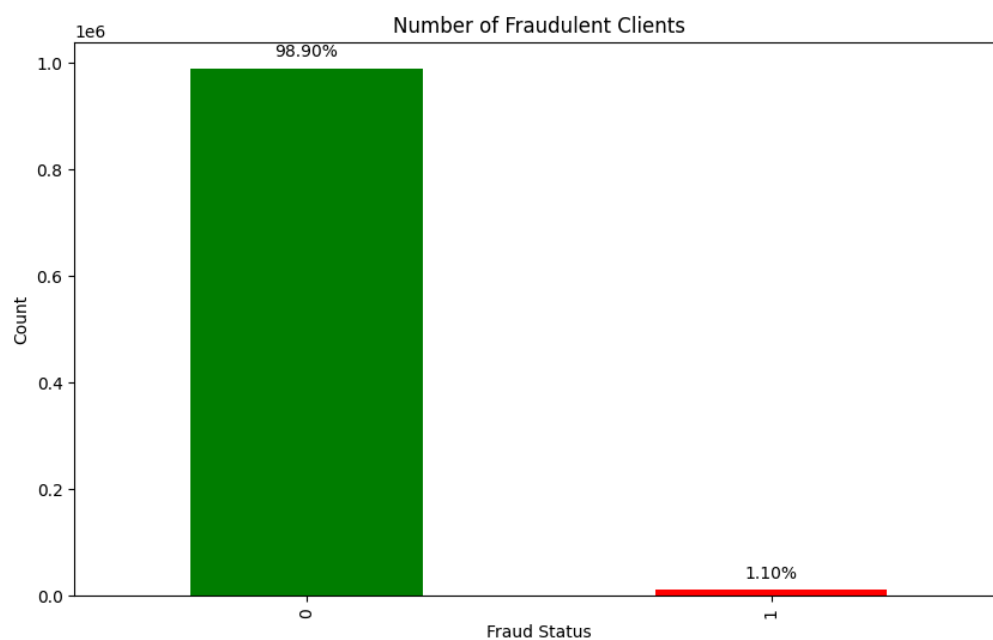


Fig.1: Distribution of Clients by Fraud Status

Class distribution is highly imbalanced, with fraudulent cases making up a small minority of the data. This reflects real-world conditions and presents a meaningful challenge for model development.

3.4 Data Collection and Validation

The BAF dataset was generated using synthetic data techniques based on a real-world dataset of fraudulent bank account applications. It was created using a CTGAN, trained on anonymised real data to simulate realistic fraud detection scenarios while ensuring user privacy. The dataset was curated by generating approximately 2.5 million synthetic records, from which a final 1 million instances were sampled using a bias-aware sampling strategy. This sampling considered both the presence of protected attributes (such as age group, employment status, and income percentile) and fraud labels to maintain underlying statistical properties while enabling fairness and bias evaluation. The dataset also includes a temporal feature (month) to support time-aware evaluation, allowing researchers to analyse distributional drift and assess how fraud detection models degrade over time.

Despite these efforts, several potential sources of error and bias are inherent in the dataset. Selection bias is introduced due to upstream business and regulatory filters, such as anti-money laundering constraints, which shape the population of applicants included in the original dataset. Synthetic generation bias may also arise, as CTGANs may introduce sampling artefacts or fail to capture rare edge-case fraud behaviours. Labelling bias is another concern: while confirmed fraud cases are generally reliable, there may be false positives among legitimate applications that were incorrectly flagged. Moreover, the presence of temporal distribution shifts means models trained in earlier months may perform poorly in future months due to changing fraud strategies. Certain fields also introduce synthetic noise or serve as proxies for missing values, such as placeholder values like -1 or negative values in certain numerical fields, which must be treated carefully during preprocessing.

Additionally, because the CTGAN is a generative model, it may introduce unrealistic combinations of feature values, or overfit to particular patterns found in the source data. Consequently, while the BAF dataset is a valuable tool for benchmarking fraud detection systems and fairness-aware modelling, it is not intended for use in production environments. Instead, it serves as a testbed for experimental model development, bias mitigation, and temporal robustness analysis under controlled yet realistic conditions.

4. Methods

4.1 Data Cleaning and Pre-Processing

To prepare the BAF dataset for machine learning modelling, a structured data cleaning and preprocessing pipeline was implemented. The raw dataset (Base.csv) consisted of one million rows and 32 columns, including both numerical and categorical variables. An initial inspection using `.info()` and `.describe()` confirmed that there were no NaN values, but placeholder values such as -1 and negative numbers were used to encode missing or invalid

entries. A check for duplicated rows revealed that the dataset contained no duplicates. Additionally, the number of unique values in each column was assessed to guide downstream encoding decisions.

A number of features contained encoded missing values or out-of-range entries that required cleaning. Features *prev_address_months_count* and *current_address_months_count* used -1 to indicate missing information. Two binary indicator columns, *prev_address_missing* and *current_address_missing*, were created to retain this information. These -1 values were then replaced with the median value of each respective feature. Similarly, values less than zero in *intended_balcon_amount*, which reflected invalid or missing entries, were replaced with the column median. Additional fields such as *bank_months_count*, *session_length_in_minutes*, and *device_distinct_emails_8w* were also cleaned. For continuous features, the -1 values were replaced with the median while the count-type variables were replaced to 0. After these corrections, all features were re-verified to ensure that no invalid placeholders remained.

Exploratory data analysis was conducted to better understand feature distributions and fraud patterns. Histograms were plotted for all numerical features to visualise their distributions and detect outliers. For categorical features, count plots were generated with a breakdown by fraud status and log-scaled y-axes to accommodate extreme skew.

4.2 Feature Engineering

To enhance the model's ability to detect fraudulent applications, we engineered several new features that captured latent patterns in user behaviour and financial characteristics.

Table 2: Engineered Features

	Engineered Feature	Purpose
1	$\text{st_velocity_ratio} = \text{velocity_6h} / (\text{velocity_24h} + 1)$	Captures short-term transaction intensity relative to daily activity to detect sudden bursts of transaction behaviour in the last 6 hours, which is often a sign of fraudulent activity executed quickly before being flagged.
2	$\text{lt_velocity_ratio} = \text{velocity_24h} / (\text{velocity_4w} + 1)$	Measures how active the user is in the last 24 hours compared to their longer-term behaviour over 4 weeks, which is useful for flagging unusual spikes in activity from users who are otherwise low-frequency.
3	$\text{total_address_months} = \text{prev_address_months_count} + \text{current_address_months_count}$	Frequent address changes might indicate fraud.
4	$\text{income_bin} \rightarrow$ Categorised income into Low, Medium, and High	To improve interpretability and help tree-based models perform better.
5	$\text{credit_risk_category} \rightarrow$ Categorised credit_risk_score into Low, Medium, High	

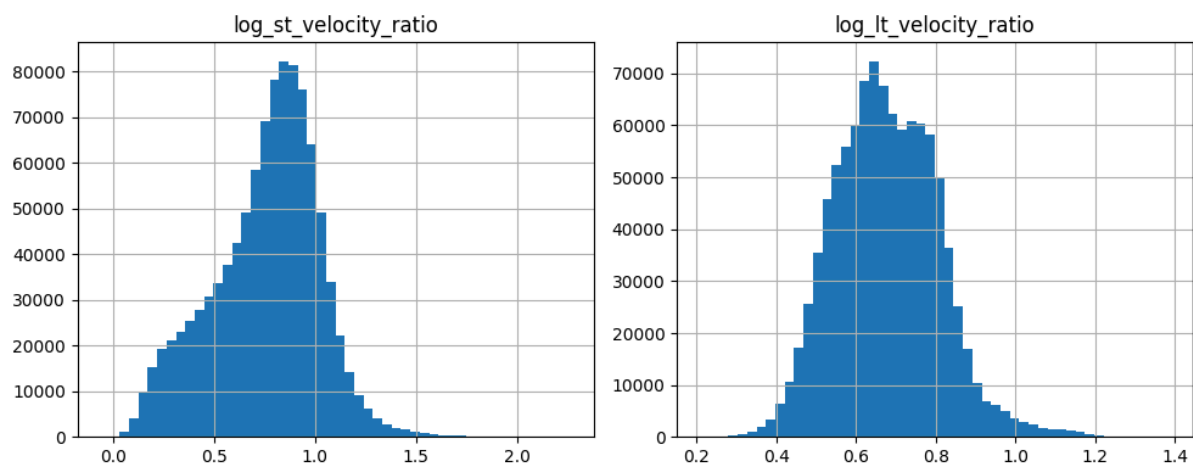


Fig. 2: Histogram of log-transformed short-term and long-term velocity ratios

These log-transformed features exhibited well-behaved, near-normal distributions, making them more suitable for linear modeling techniques like logistic regression.

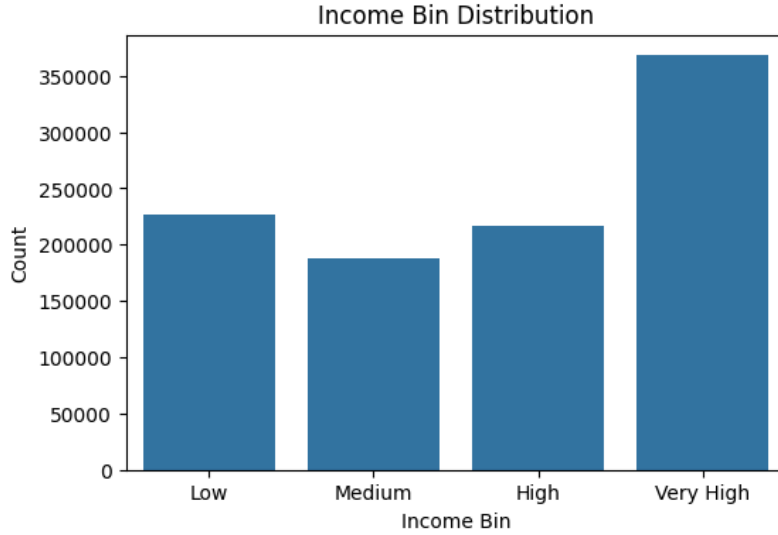


Fig. 3: Distribution of Income Bin Feature

This new `income_bin` feature improved interpretability and reduced the model's sensitivity to income outliers. Interestingly, fraud was most prevalent in the Very High income group (1.74%), suggesting that fraudsters may intentionally present themselves as affluent to gain higher credit approval.

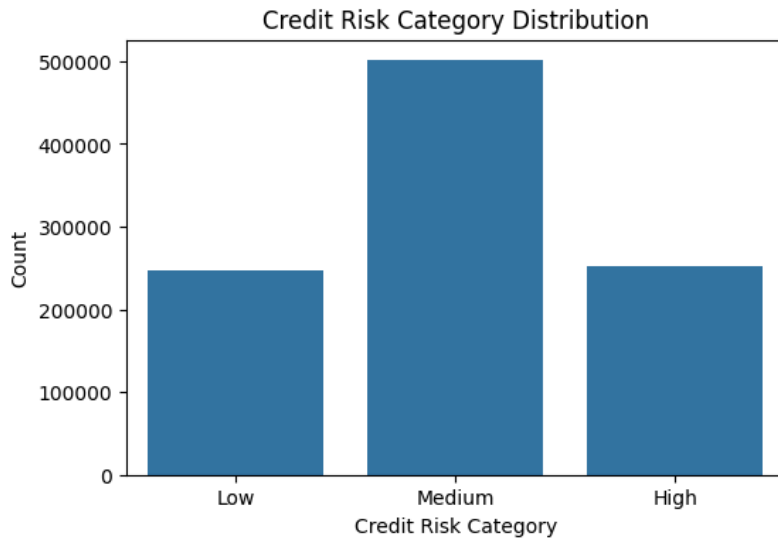


Fig. 4: Distribution of Credit Risk Category Feature

Similarly, the fraud rate was highest among High-risk applicants (2.20%), which aligns with expectations that poorly-rated profiles are more likely to exhibit fraudulent behaviour.

4.3 Encoding

Categorical variables were prepared for modelling through a combination of label encoding and one-hot encoding. Binary categorical features such as `source`, `email_is_free`, and `foreign_request` were label encoded. Multi-class categorical variables such as `payment_type`, `employment_status`, `housing_status`, `device_os`, `income_bin`, and `credit_risk_category` were one-hot encoded with the first category dropped to prevent

multicollinearity. After encoding, the training and testing sets were aligned to ensure consistent feature sets across both partitions.

4.4 Resampling Methods

To address the extreme class imbalance inherent in the dataset, we adopted the SMOTE. This technique was applied only to the training partitions of each fold during the 5-fold stratified cross-validation process.

Each training fold originally contained approximately 720,000 records, with a fraud prevalence of just 1.06%. After applying SMOTE, the minority class (fraud) was oversampled to match the majority class, resulting in resampled training sets of approximately 1.43 million records with a balanced 50:50 fraud-to-non-fraud ratio. The validation folds, by contrast, were left untouched to preserve their naturally imbalanced distribution, enabling realistic model evaluation.

This approach ensures that the models are trained on a balanced representation of both classes, helping them learn meaningful fraud patterns without being biased toward the dominant class. At the same time, performance evaluation on the original imbalanced distribution provides a fair and practical assessment of how the model would perform in a real-world deployment setting.

The choice of 5-fold cross-validation strikes a practical balance between training efficiency and model reliability. It allows the model to be evaluated on multiple partitions of unseen data without excessive computational cost. Given the size of the dataset, increasing the number of folds would be computationally expensive with diminishing returns, while using fewer folds would risk higher variance in validation performance. Thus, 5-fold validation was chosen to provide robust performance estimates, reduce overfitting risk, and enable generalisation testing across diverse temporal segments of the data.

This setup ensures that models are trained on balanced data while being evaluated on realistic, imbalanced samples. Additionally, saving each fold's resampled training data and untouched validation data separately facilitates reproducibility and consistent evaluation across different modelling approaches.

4.5 Test/Train Split, Holdout and Validation Approach

To ensure robust model evaluation and avoid data leakage, a temporal split strategy was adopted using the *month* feature available in the dataset. Specifically, data from the first six months (months 0 to 6) were used for training the model, while data from the last two months (months 7 and 8) were reserved for testing. This approach mirrors a real-world deployment setting where past data is used to predict future outcomes, enabling evaluation under realistic distribution shift scenarios.

The resulting split yielded 903,157 records for training and 96,843 records for testing. The target variable for this task is *fraud_bool*, a binary indicator of whether an application was fraudulent (1) or legitimate (0). *X_train* and *X_test* were separated from the target labels, *y_train* and *y_test*, prior to model training and evaluation.

4.6 Exploratory Data Analysis Insights

We observed a meaningful difference in fraud rates across digital access channels. Applications submitted through TELEAPP (mobile application) had a relatively higher proportion of fraud cases compared to those from INTERNET (web browser). This may suggest that fraudsters exploit the relative ease or anonymity of mobile channels.

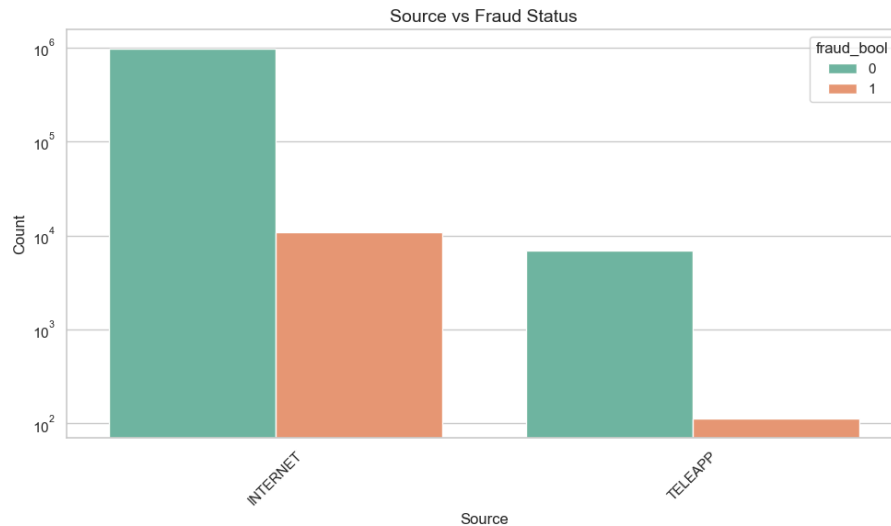


Fig. 5: Relationship between Digital Access Channels and Fraudulent Applications

Another significant observation was tied to the device operating system. While most applications came from Windows or Linux systems, fraud appeared more concentrated among devices using obscure OS labels like X11, other, or Macintosh, possibly indicating spoofed environments or virtual machines.

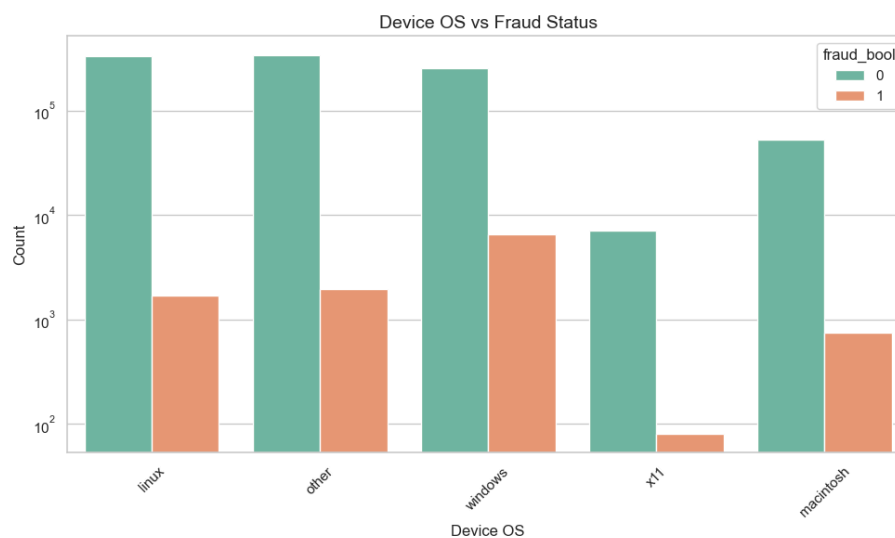


Fig. 6: Relationship between Device OS and Fraudulent Applications

4.6.1 Hypothesis 1: Missing address history is indicative of higher fraud risk

One of the most compelling findings was that fraud cases were disproportionately associated with missing or minimal address histories. Applicants with missing values in

prev_address_missing and *current_address_missing* showed a noticeably higher fraud rate. This supports the notion that fraudsters tend to provide minimal verifiable background information.

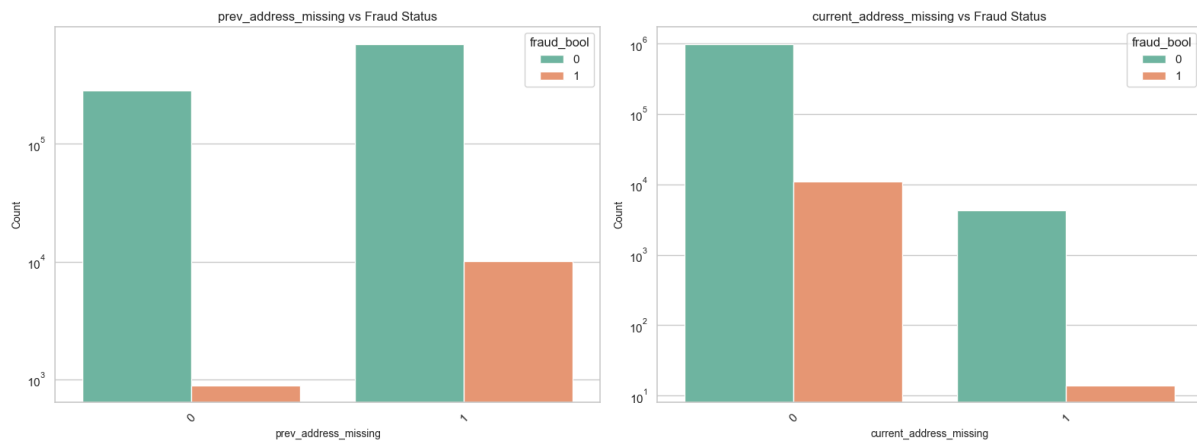


Fig. 7: Relationship between Missing Address Information and Fraudulent Applications

We engineered binary flags for missing address information and explicitly retained them in our models. These features are especially useful for tree-based models like Random Forest and XGBoost, which can capture their interactions with behavioural variables such as application velocity. In deployment, these address completeness flags can also be used by business teams to trigger additional KYC (Know Your Customer) verification steps or manual reviews.

4.6.2 Hypothesis 2: Abnormally high short-term velocity suggests automated or coordinated fraud behaviour.

Among the numerical features, *velocity_6h* and *velocity_24h* are right-skewed, especially *velocity_6h*, which indicates that most applicants have relatively low recent activity, but a small number exhibit extremely high submission rates. The spike in density around the 4,000 to 6,000 range suggests clustered behaviours, potentially representing coordinated application attempts.

Unlike the 6h and 24h velocities, *velocity_4w* displays a highly irregular, multimodal distribution. The spikes suggest manual overrides, threshold-driven limits, or discrete system-triggered behaviours such as batch processing or cut-off limits. These discrete jumps may hint at system-generated patterns or suspicious behaviours occurring cyclically, which are valuable for temporal fraud modelling.

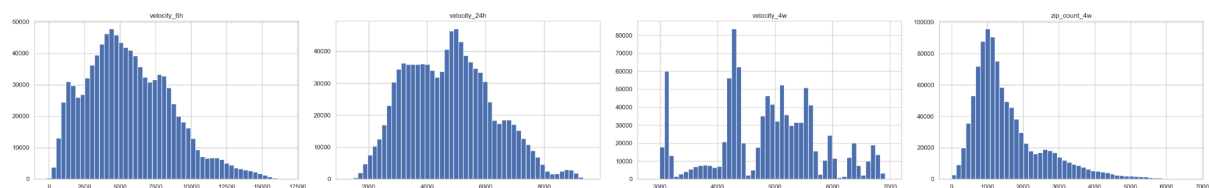


Fig. 8: Distribution of Velocity Features

This insight led us to engineer two new features: *st_velocity_ratio* and *lt_velocity_ratio*, which quantify sudden short-term and long-term submission spikes relative to baseline patterns. These ratios help the model detect anomalous surges in application activity that may indicate coordinated or automated fraud attempts. Additionally, we applied log transformations to reduce skew and improve model learning. Tree-based models like XGBoost are particularly well-suited to capture threshold effects in these engineered ratios. By incorporating these velocity-based features, we aim to strengthen the model's ability to detect burst fraud campaigns without relying solely on raw counts.

4.6.3 Hypothesis 3: Higher income applicants are more likely to be fraudulent.

While it's often assumed that fraud risk is concentrated in lower-income segments, our exploratory analysis reveals that fraud may be more prevalent among applicants reporting the highest income levels. A possible rationale is that fraudsters may intentionally inflate income to bypass automated credit risk checks or secure higher-value credit lines.

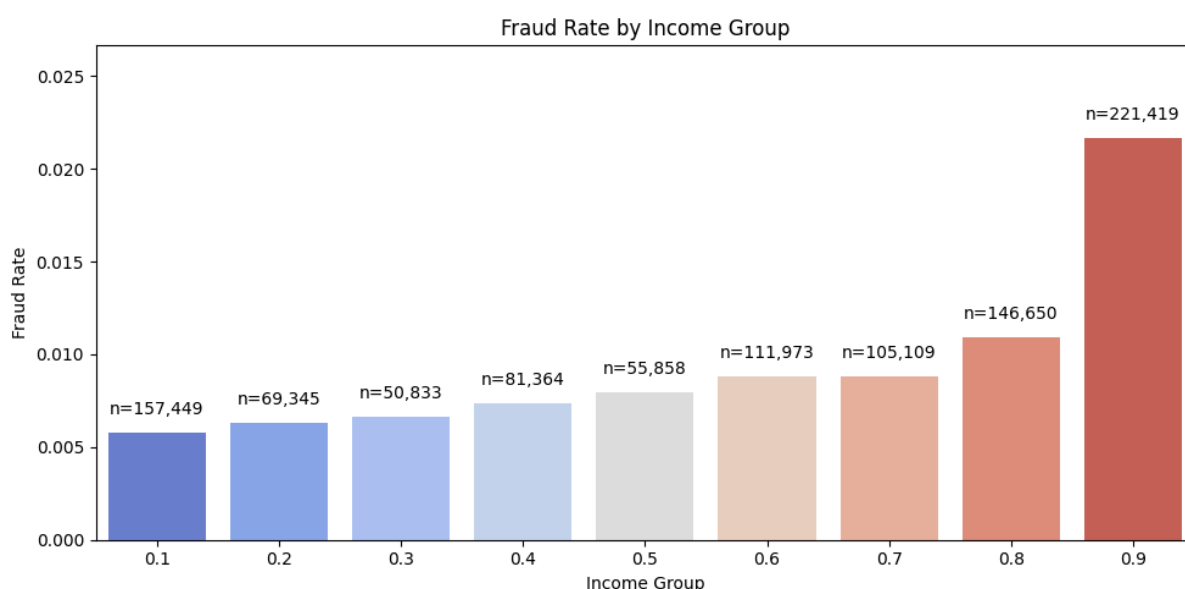


Fig. 9: Fraud Rate by Income Group

The fraud rate by income group reveals an unexpected pattern. Fraud rates increase significantly in the highest income decile (0.9), which has over 2.5% fraud rate, compared to under 1% in other groups. This suggests that fraudsters may be strategically inflating income fields to appear more creditworthy. High-income claims may also attract larger credit limits or promotional benefits, increasing the reward for successful fraud.

This insight directly informed our feature engineering approach. We created a new categorical feature, *income_bin*, to segment applicants into Low, Medium, High and Very High income tiers. This binning allows the model to detect non-linear relationships between income levels and fraud, especially the elevated risk associated with the highest tier. It also improves interpretability and reduces noise from treating income as a continuous variable.

4.7 Baseline Model

To establish a benchmark for model comparison, a Logistic Regression classifier was implemented as the baseline model. Logistic regression is a commonly used linear model in binary classification tasks due to its simplicity, interpretability, and relatively low computational cost. It serves as a strong starting point for evaluating more complex models in fraud detection pipelines.

The model was trained using a combined training set consisting of folds 1 through 4, which had been resampled using SMOTE to address class imbalance. Fold 5 was held out as a validation set to ensure unbiased performance evaluation. All input features were scaled using *StandardScaler* to ensure fair contribution of each feature, given that logistic regression is sensitive to feature magnitude.

```
Classification Report:
              precision    recall  f1-score   support

     0       0.9896      0.9910      0.9903      178711
     1       0.0301      0.0260      0.0279         1920

 accuracy      0.9807      180631
 macro avg      0.5098      0.5085      0.5091      180631
 weighted avg      0.9794      0.9807      0.9800      180631

Confusion Matrix:
[[177100  1611]
 [ 1870    50]]

Precision: 0.0301
Recall:    0.0260
F1 Score:  0.0279
AUC-PR (Average Precision): 0.0141
ROC AUC:   0.5542
```

Fig. 10: Classification Report and Evaluation Metrics for Baseline Logistic Regression

Upon evaluation, the model achieved an overall accuracy of 98.07%, but this figure is misleading due to the class imbalance. A closer look at class-specific metrics reveals the model's limitations in detecting fraudulent cases. The model attained a precision of 3.01%, recall of 2.60%, and an F1 score of 2.79% for the fraud class. The Area Under the Precision-Recall Curve (AUC-PR) was notably low at 0.0141, suggesting poor ability to differentiate fraudulent cases from legitimate ones across probability thresholds. The ROC AUC was 0.5542, only marginally better than random guessing.

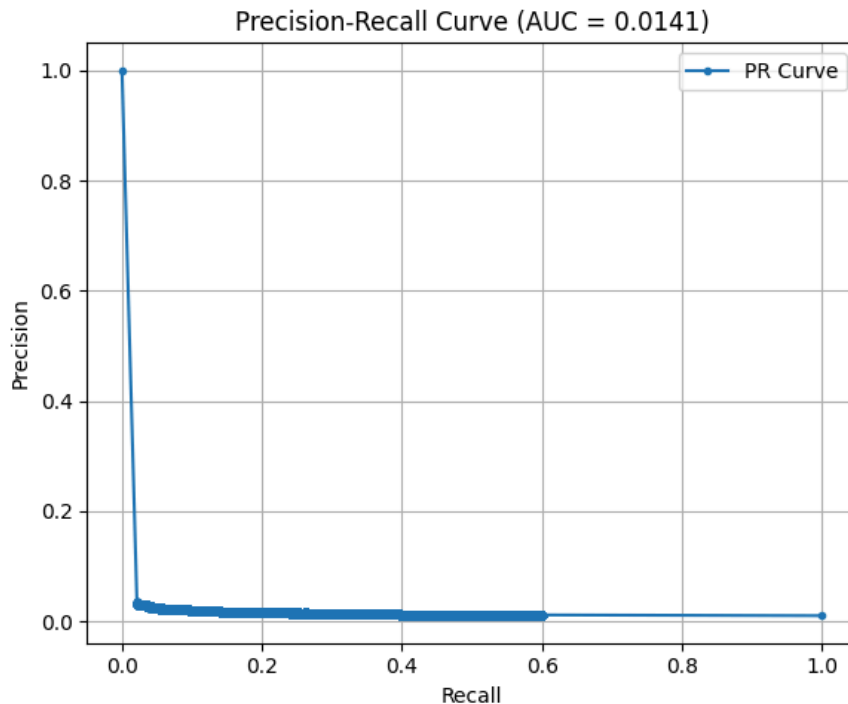


Fig.11: Precision-Recall Curve for Baseline Logistic Regression

The PR curve further illustrates the model's struggles. Precision quickly plummets, reflecting the model's inability to maintain high precision as recall increases. This outcome highlights the inherent challenge of fraud detection in highly imbalanced datasets and justifies the need for more sophisticated ensemble approaches that can better capture subtle fraud patterns.

4.8 Challenger Models

To expand upon the baseline logistic regression, several challenger models were explored, namely XGBoost, Random Forest, RNN-GRU, and Naive Bayes. Each model was selected based on its suitability for different data structures and its track record in fraud detection or classification problems.

- XGBoost is a gradient boosting framework that builds an ensemble of weak learners to iteratively minimise classification error through optimised decision tree construction.
- Random Forest is an ensemble learning method that constructs multiple decision trees via bootstrapping and averages their outputs to improve generalisation and reduce overfitting.
- RNN-GRU is a type of recurrent neural network that captures sequential dependencies using gated mechanisms, commonly used in time-series and sequence modelling tasks.
- Naive Bayes is a probabilistic classifier based on Bayes' theorem that assumes feature independence, offering fast training and interpretability on high-dimensional data.

While XGBoost and Random Forest demonstrated robust and reliable performance in fraud classification, two other models (RNN-GRU and Naive Bayes) were ultimately excluded from the final pipeline due to consistently underwhelming results during evaluation.

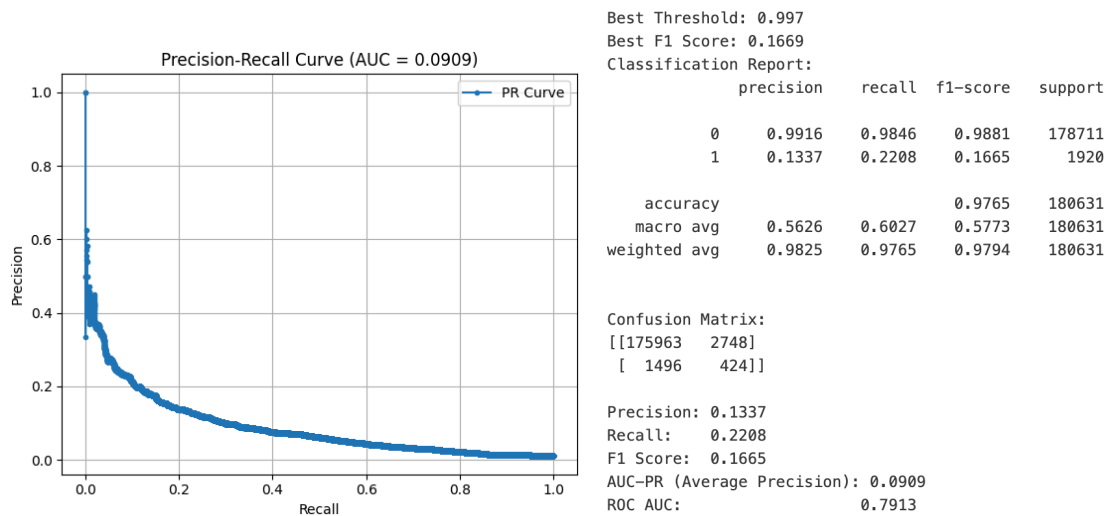


Fig. 12: Precision-Recall curve and performance metrics for the Naive Bayes classifier

Naive Bayes performed poorly due to its assumption of feature independence, failing to capture key interactions in fraud detection. It struggled with temporal, categorical, and behavioral signals, yielding low precision and recall for fraud (Fig. 24), and lacked adaptability to temporal shifts in the BAF dataset.

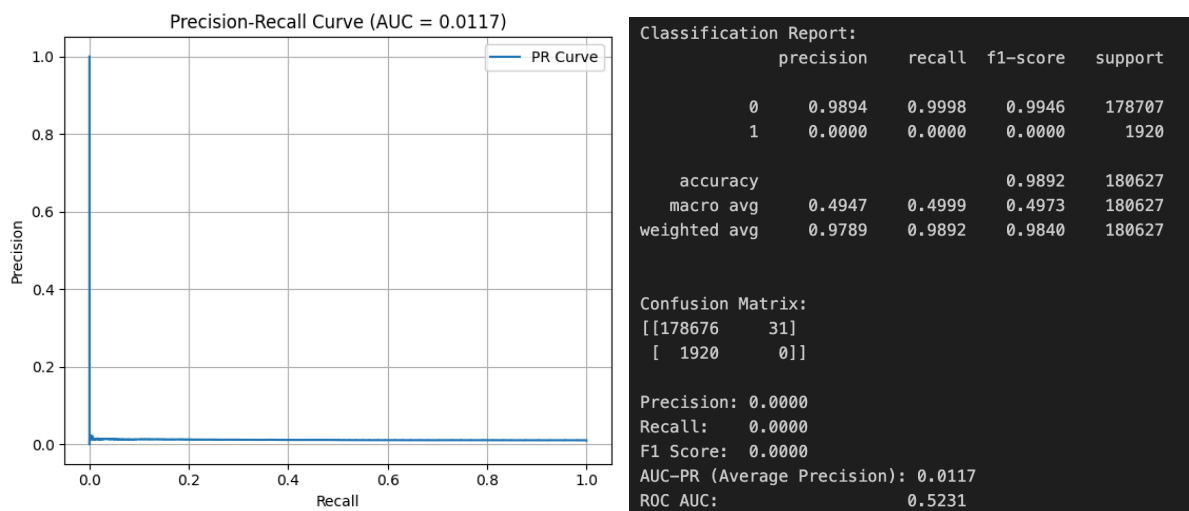


Fig. 13: Precision-Recall curve and performance metrics for GRU

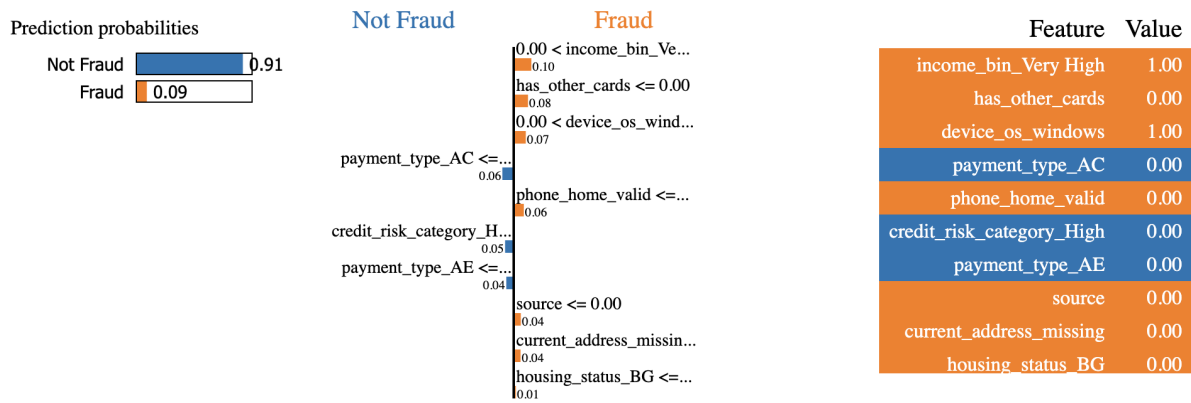


Fig. 14: LIME explanation for the poor performance of GRU

LIME analysis (Fig. 14) showed that the GRU model missed key fraud indicators across all three hypotheses. It underreacted to missing address data (Hypothesis 1), underweighted mobile sources like TELEAPP (Hypothesis 2), and misclassified high-income applicants as low-risk (Hypothesis 3). Despite LIME surfacing relevant signals, the model's evaluation metrics (Fig. 13) confirm its poor suitability for this task.

Importantly, all challenger models were evaluated and tuned based on the AUC-PR. While ROC-AUC is a common metric, it becomes less informative in highly imbalanced scenarios, where the overwhelming majority of true negatives can mask poor fraud recall performance.

In our case, although the training sets were balanced using SMOTE (50:50 fraud-to-non-fraud ratio), the validation datasets remained naturally imbalanced, reflecting real-world conditions where fraud is rare. AUC-PR more accurately reflects model performance on the minority fraud class, as it focuses on the trade-off between precision and recall without being distorted by true negatives.

4.8.1 Gradient Boosting

To evaluate a more advanced ensemble learning method for fraud detection, an XGBoost classifier was implemented. The model was trained on folds 1 - 4 of the SMOTE-resampled dataset, with fold 5 used exclusively for validation. While feature scaling was not applied (as XGBoost is a tree-based method and invariant to feature magnitude), feature selection was conducted to improve model interpretability and performance.

To account for the inherent class imbalance, the *scale_pos_weight* parameter was calculated as the ratio of negative to positive samples in the training set. This weight was used during hyperparameter tuning to help the model better recognise minority class instances (fraud cases).

The model's predicted probabilities on the validation set were used to identify the optimal classification threshold that maximised the F1 score on the fraud class. This threshold tuning step proved critical in achieving a balance between precision and recall.

Final Evaluation:

Using the optimised threshold of **0.329**, the final tuned XGBoost model achieved:

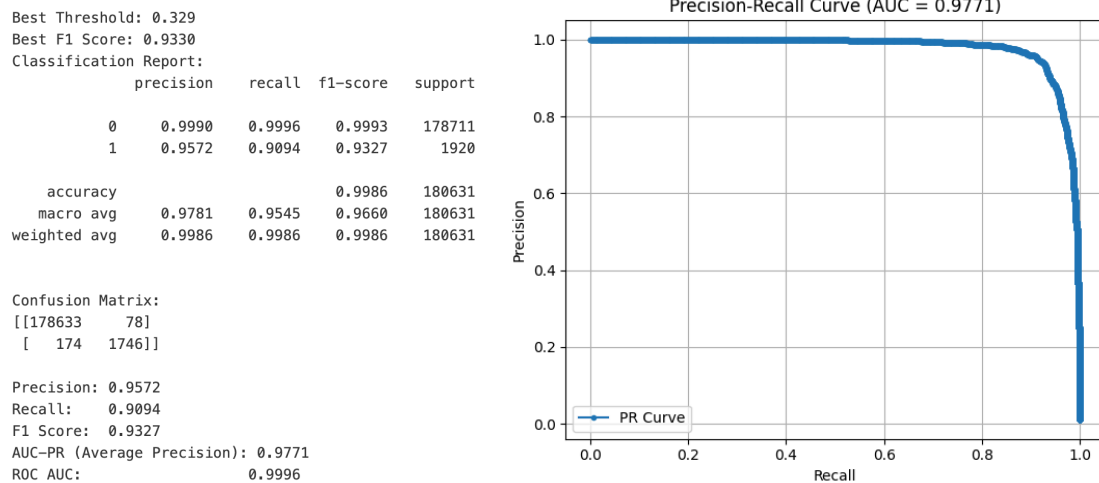


Fig.15: Precision-Recall Curve and performance metrics for Gradient Boosting

Compared to the Logistic Regression baseline (F1: 0.0279, ROC AUC: 0.5542, AUC-PR: 0.0141), the XGBoost model achieved **dramatically higher performance across all metrics**. It correctly identified **1,746 fraudulent cases** with high precision, a substantial improvement over both Logistic Regression (50) and Naive Bayes (1,127 but with much lower precision).

These results showcase **XGBoost's ability to capture complex, non-linear feature interactions**, making it a powerful tool for fraud detection in highly imbalanced datasets. The near-perfect ROC AUC and AUC-PR scores highlight its strong discrimination capability and robustness, firmly establishing XGBoost as a superior alternative to traditional linear models in this context.

4.8.2 Random Forest

No feature scaling was done as random forest models are inherently robust to features with different scales.

First, a baseline Random Forest Classifier (baseline_rf) was trained on folds 1-4 with default parameters and all features. Evaluation was done on the unseen fold 5 validation dataset.

```

Classification Report:
              precision    recall  f1-score   support

     0       0.9934       0.9519       0.9722    178711
     1       0.0835       0.4083       0.1387     1920

 accuracy          0.9461    180631
 macro avg       0.5385    0.6801    0.5554    180631
 weighted avg    0.9837    0.9461    0.9633    180631

Confusion Matrix:
[[170111  8600]
 [ 1136   784]]

Precision: 0.0835
Recall:    0.4083
F1 Score:  0.1387
AUC-PR (Average Precision): 0.0891
ROC AUC:           0.8546

```

Fig.16: Results of baseline_rf

To try reducing dimensionality and minimise overfitting, we chose an optimal number of features based on cumulative importance. We went ahead with the top 20 features based on feature importance from the baseline RF model. Subsequently, we performed tuning with Optuna on 10% of folds 1-4 to test for the best combination of hyperparameters. Our Optuna function searches for hyperparameters which maximise AUC-PR. With the best hyperparameters, the model was trained on folds 1-4 of the SMOTE-resampled dataset. Then validated on the unseen fold 5 data to give evaluation metrics. (rf_model_2)

To have a better view of whether feature selection improved performance of the model, rf_model_3 was trained on all features, keeping other methods the same as rf_model_2.

```

Classification Report:
              precision    recall  f1-score   support

     0       0.9926       0.9698       0.9810    178711
     1       0.1035       0.3250       0.1570     1920

 accuracy          0.9629    180631
 macro avg       0.5480    0.6474    0.5690    180631
 weighted avg    0.9831    0.9629    0.9723    180631

Confusion Matrix:
[[173307  5404]
 [ 1296   624]]

Precision: 0.1035
Recall:    0.3250
F1 Score:  0.1570
AUC-PR (Average Precision): 0.0982
ROC AUC:           0.8997

```

Fig.17: Results of rf_model_2

```

Classification Report:
              precision    recall  f1-score   support

      0       0.9933      0.9674      0.9802    178711
      1       0.1143      0.3917      0.1770      1920

   accuracy      0.9839      0.9613      0.9716    180631
  macro avg       0.5538      0.6795      0.5786    180631
 weighted avg       0.9839      0.9613      0.9716    180631

Confusion Matrix:
[[172886   5825]
 [  1168    752]]

Precision: 0.1143
Recall:    0.3917
F1 Score:  0.1770
AUC-PR (Average Precision): 0.1131
ROC AUC:           0.9085

```

```

RandomForestClassifier(max_depth=14, max_features='log2', min_samples_leaf=2,
                      min_samples_split=3, n_estimators=85, n_jobs=-1,
                      random_state=42)

```

Fig.18: Results & Optuna Hyperparameters of rf_model_3

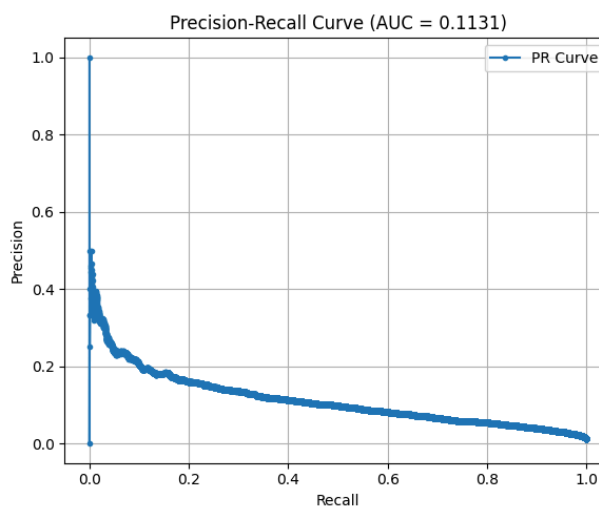


Fig.19: Precision-Recall Curve for rf_model_3

Compared to the Logistic Regression baseline (F1: 0.0279, ROC AUC: 0.5542, AUC-PR: 0.0141), Random Forest outperforms LogReg (Logistic Regression) by a large margin in precision, recall, F1-score, and AUC-PR.

AUC-PR for rf_model_3 (0.1131) showed a modest improvement from rf_model_2 (0.0982), indicating that rf_model_3 can differentiate fraud and non-fraud better. Recall and F1-score also saw marginal improvement from rf_model_2 to rf_model_3. Given our context of fraud detection, rf_model_3 with higher recall and higher AUC-PR yielded the best model evaluation results.

Despite improvements in evaluation metrics, `rf_model_3` still struggles with low recall. `rf_model_3` misses out 1168 fraud cases while correctly identifying 752 as fraud. Feature selection was not enough to significantly improve model generalisation. Class imbalance still poses a challenge to the random forest model even after SMOTE, feature selection, and balancing class weights. Expanding the hyperparameter search space and applying advanced models like XGBoost may give better results.

4.8.3 Model Ensemble: Blended Bagging and Boosting

To improve predictive performance beyond baseline models, we implemented a blended ensemble combining the strengths of bagging (Random Forest) and boosting (XGBoost). This hybrid approach leverages the variance-reducing properties of Random Forest with the high bias-reducing capabilities of XGBoost, offering a robust fraud detection model. Initially, both models were trained on the combined resampled folds 1 to 4 via SMOTE, and predictions were averaged using an equal-weighted soft voting scheme. Evaluation on fold 5 yielded seemingly outstanding metrics, including a perfect AUC-PR of 1.0000, precision of 1.0000, and F1-score of 0.9149. While these results were promising, they appeared too optimistic and raised concerns about potential overfitting or data leakage.

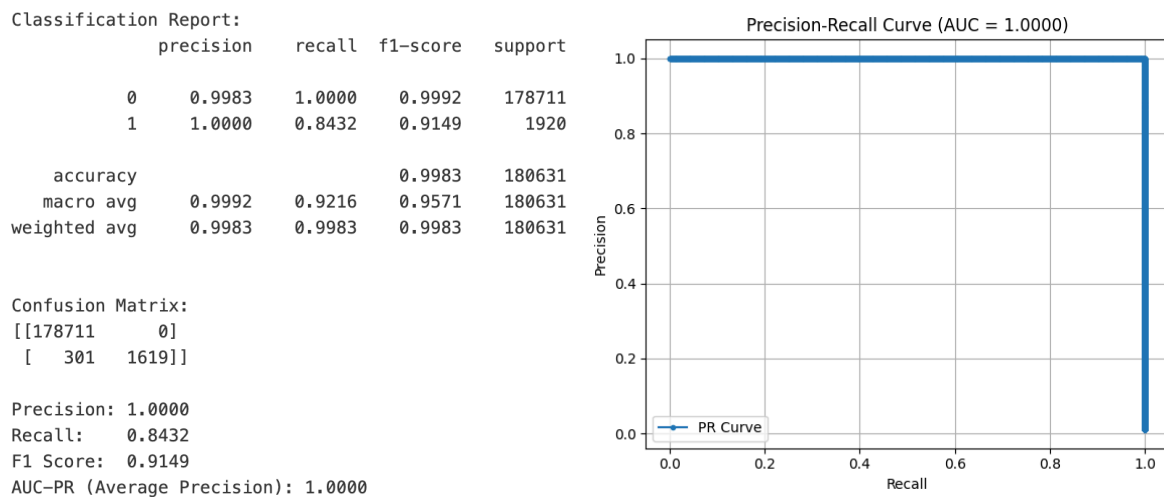


Fig.20: Baseline Evaluation Metrics of Model Ensemble

Upon further inspection, we identified two major limitations in the initial setup. First, the ensemble was trained and evaluated using only a single holdout fold (fold 5), which can lead to biased metrics if that fold happens to be easier or unrepresentative. Second, hyperparameters and thresholds were tuned on the same fold used for evaluation, violating the principle of fair testing and inflating performance metrics. To rectify this, we adopted a more rigorous cross-validation-based ensemble training pipeline using folds 1 to 4.

To address this, we adopted a more robust ensemble validation protocol involving full 5-fold cross-validation and Optuna hyperparameter tuning across multiple folds. Specifically, we repeated the ensemble training process for folds 1 to 4, using three folds for training and one for validation in each iteration. For each fold, we used Optuna to tune key hyperparameters of both the RFC and XGB models, as well as the blending weight between them.

The key objective in the Optuna tuning process was to maximise the AUC-PR on the validation fold.

To reduce computation time while retaining class balance, we sampled 1 million rows from the SMOTEd training data (50:50 fraud to non-fraud) in each fold for tuning. This size was selected to match the scale of the original dataset before oversampling, ensuring a fair representation of the feature space without unnecessary overhead.

After tuning was complete for each fold, we averaged the optimal hyperparameters across folds to obtain a single set of robust parameters for both classifiers and the blending weight. These final parameters were then applied to train the RFC and XGB models on folds 1 to 4 combined, with evaluation performed on the holdout fold 5. To avoid bias from using a fixed threshold, we also performed threshold tuning, selecting the probability cutoff that maximised the AUC-PR on the validation set.

```
Averaged Hyperparameters Across Folds:  
Random Forest - n_estimators: 188  
Random Forest - max_depth: 14  
Random Forest - min_samples_split: 11  
XGBoost - n_estimators: 246  
XGBoost - max_depth: 11  
XGBoost - learning_rate: 0.20785215049521782  
XGBoost - subsample: 0.8061277675780374  
XGBoost - colsample_bytree: 0.6514724551599868  
Blend Weight: 0.6749003261676731
```

Fig.21: Averaged Hyperparameters Across Optuna

The averaged hyperparameters derived from the cross-validation process offer meaningful insight into the structure and behaviour of the final ensemble model. For the Random Forest classifier, the optimal configuration consisted of 188 estimators with a maximum tree depth of 14 and a minimum sample split threshold of 11. These settings balance model complexity and generalisation by allowing moderately deep trees while limiting excessive branching that could lead to overfitting. For the XGBoost classifier, the model used 246 boosting rounds, a maximum depth of 11, and a moderately high learning rate of 0.208. Together with a subsample rate of 0.806 and a column subsampling rate of 0.651, this configuration ensures that each tree learns from a diverse subset of data and features, reducing overfitting and improving model robustness. Most notably, the tuned blending weight was 0.675 in favor of XGBoost, indicating that the boosting model contributed more heavily to the ensemble's predictions. This suggests that XGBoost was more effective in learning complex fraud patterns within the dataset, while the inclusion of Random Forest provided additional variance reduction and ensemble diversity.

```

Classification Report:
              precision    recall  f1-score   support

     0       0.9999       0.9999       0.9999       178711
     1       0.9911       0.9865       0.9888         1920

 accuracy      0.9998      0.9998      0.9998      180631
  macro avg   0.9955      0.9932      0.9943      180631
 weighted avg 0.9998      0.9998      0.9998      180631

Confusion Matrix:
[[178694   17]
 [    26 1894]]

Precision: 0.9911
Recall:    0.9865
F1 Score:  0.9888
AUC-PR (Average Precision): 0.9984

```

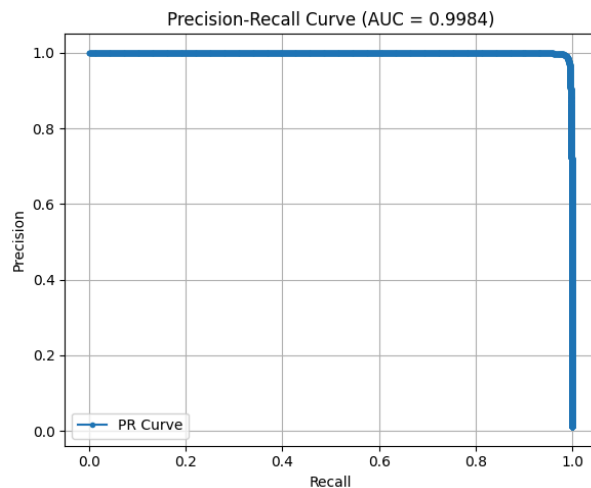


Fig.22: Evaluation Metrics of Model Ensemble after Optuna

Compared to the baseline ensemble model which achieved perfect or near-perfect metrics including an AUC-PR of 1.0000 and F1-score of 0.9177, the final model demonstrated a slight but expected decline in performance after applying proper cross-validation and avoiding evaluation leakage. This drop is largely attributable to the removal of data leakage risks and the adoption of more realistic validation practices. Specifically, the baseline model was tuned and evaluated on the same holdout fold (fold 5), which likely led to overfitting and inflated metrics. In contrast, the final model used robust 5-fold cross-validation and separate tuning/evaluation sets, resulting in more honest and generalizable performance. Despite this, the final model still achieved an outstanding AUC-PR of 0.9984 and an F1-score of 0.9888, reflecting strong generalisation ability and confirming that the ensemble design and feature engineering strategies were effective even under stricter validation protocols. This outcome validates the use of a blended ensemble and demonstrates its reliability in detecting fraud beyond a single fold.

5. Results

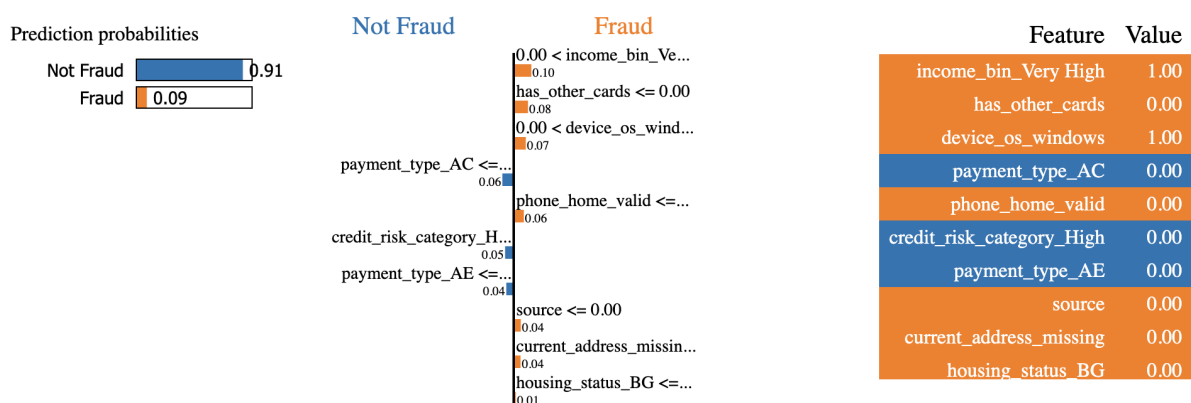


Fig. 23: Best LIME Result for Model Ensemble ($X_{val.iiloc[0]}$)

Model Ensemble					Gradient Boosting					Random Forest				
Classification Report:					Classification Report:					Classification Report:				
	precision	recall	f1-score	support		precision	recall	f1-score	support		precision	recall	f1-score	support
0	1.0000	1.0000	1.0000	178711	0	0.9990	0.9996	0.9993	178711	0	0.9933	0.9674	0.9802	178711
1	0.9969	0.9969	0.9969	1920	1	0.9572	0.9094	0.9327	1920	1	0.1143	0.3917	0.1770	1920
accuracy			0.9999	180631	accuracy			0.9986	180631	accuracy			0.9613	180631
macro avg	0.9984	0.9984	0.9984	180631	macro avg	0.9781	0.9545	0.9660	180631	macro avg	0.5538	0.6795	0.5786	180631
weighted avg	0.9999	0.9999	0.9999	180631	weighted avg	0.9986	0.9986	0.9986	180631	weighted avg	0.9839	0.9613	0.9716	180631
Confusion Matrix:					Confusion Matrix:					Confusion Matrix:				
[[178705 6]					[[178633 78]					[[172886 5825]				
[6 1914]]					[174 1746]]					[1168 752]]				
Precision: 0.9969					Precision: 0.9572					Precision: 0.1143				
Recall: 0.9969					Recall: 0.9094					Recall: 0.3917				
F1 Score: 0.9969					F1 Score: 0.9327					F1 Score: 0.1770				
AUC-PR (Average Precision): 0.9997					AUC-PR (Average Precision): 0.9771					AUC-PR (Average Precision): 0.1131				
					ROC AUC: 0.9996					ROC AUC: 0.9085				

Fig. 24: Side-by-side comparison between the 3 models

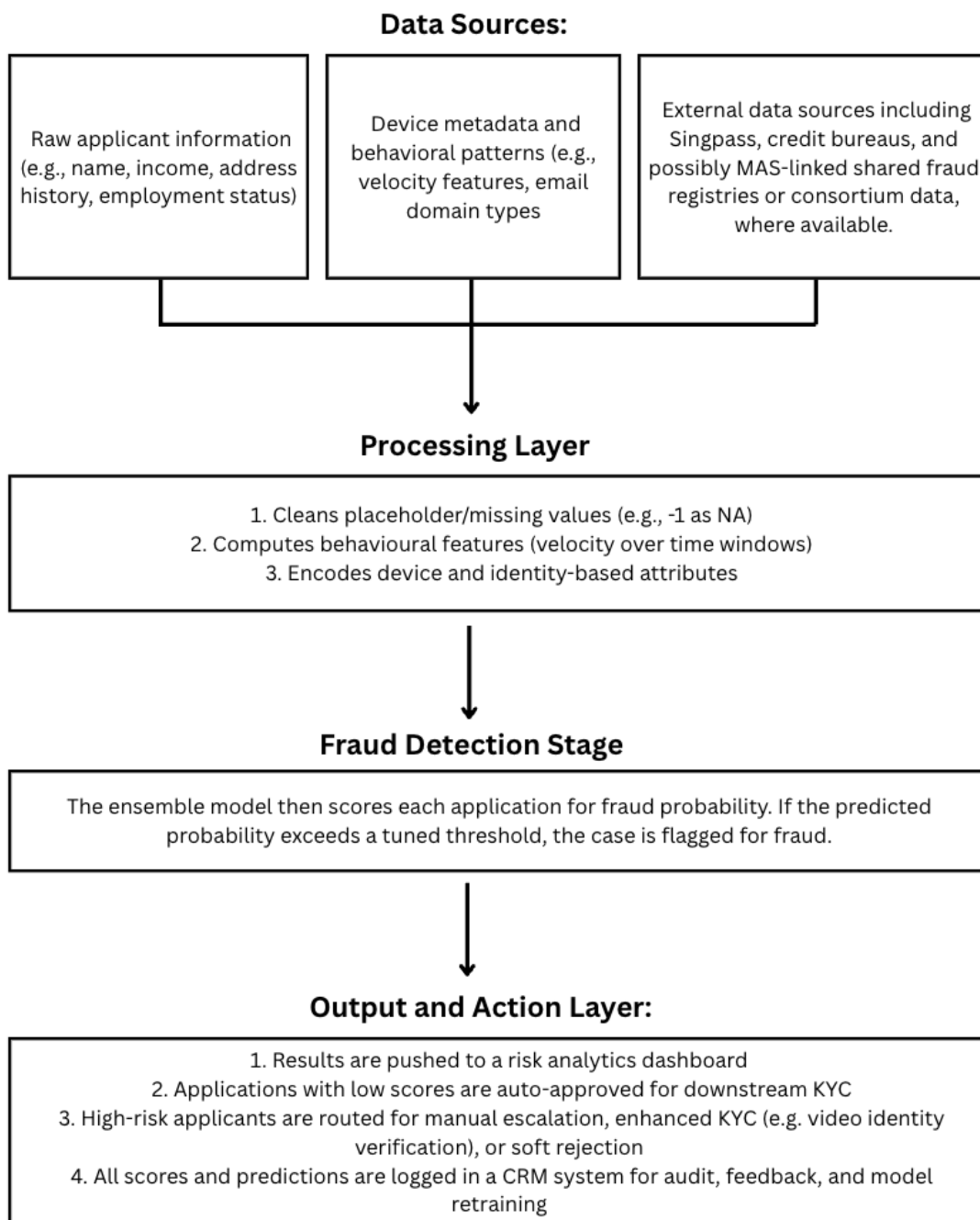
The evaluation of the models reveals that the ensemble model significantly outperforms both Gradient Boosting and Random Forest, particularly in terms of AUC-PR, the key metric maximised during hyperparameter tuning. The ensemble model achieved an AUC-PR of 0.9997, reflecting its outstanding ability to differentiate between fraudulent and non-fraudulent cases, with precision, recall, and F1 score all reaching near-perfect levels (0.9969). This contrasts sharply with the Gradient Boosting model, which, while also strong, delivered a slightly lower AUC-PR of 0.9771 despite achieving high F1 and ROC AUC scores (0.9327 and 0.9996, respectively). In contrast, Random Forest performed poorly, with an AUC-PR of 0.1259 and a low F1 score (0.1951), indicating its ineffectiveness in detecting fraud. Given that AUC-PR was prioritised during tuning, the ensemble model (Fig. 24) clearly stands out, providing the best balance of precision, recall, and overall performance in fraud detection, making it the most reliable choice for this application. Moreover, LIME was employed to explain the model's predictions, ensuring transparency and interpretability, which is crucial for real-world deployment in fraud detection systems.

6. Conclusion

By optimizing AUC-PR, the most suitable metric for imbalanced classification, we maximized the precision and recall of the model in identifying fraudulent applications. Using LIME for explanation and three domain-knowledge-based hypotheses (missing address history, velocity spikes, and high-income deception), we both verified model behavior and features significance. Given that every dollar of fraud costs financial services firms over \$4 in remediation (LexisNexis, 2024), even small improvements in early fraud detection can lead to massive cost savings, reduced reputational damage, and stronger regulatory compliance.

6.1 Hypothetical Pipeline Implementation

In a financial institution, this fraud detection model can be deployed immediately after a customer submits an account application, but before approval or Know-Your-Customer (KYC) verification. This stage is critical as it ensures that high-risk applicants are intercepted before gaining access to banking services or promotional incentives.



This setup ensures early detection of first-party fraud, reducing operational costs and preventing downstream abuse like mule accounts or bonus farming. The model could be further integrated into a modular fraud operations (FraudOps) framework with real-time alerting, appeals tracking, and human-in-the-loop review interfaces.

6.2 Recommendations for Detecting Fraud Patterns

- Sparse Identity Signals: Use *current_address_missing* and *prev_address_missing* to flag applicants with unverifiable personal history—recommend integrating with

third-party identity verification tools (e.g., Jumio, Trulioo).

- **Application Spikes:** Monitor *velocity_6h* and *velocity_24h* to detect bot-like or coordinated fraud campaigns—real-time stream monitoring via Kafka and anomaly detection via ELK Stack can help. Additionally, device fingerprinting can be used alongside third-party bot detection tools (e.g., Arkose Labs, FingerprintJS).
- **Income Misrepresentation:** Combine high income declarations with behavioural anomalies to catch fraudsters bypassing credit filters—cross-checking with payroll APIs, existing customer profiles, shared financial history or credit bureaus (e.g., Experian) can strengthen this.

7. Appendix

7.1 Data Dictionary

income: Annual income of the applicant (in decile form) (Float)

name_email_similarity: Metric of similarity between email and applicant's name. Higher values represent higher similarity (Float)

prev_address_months_count: Number of months in previous registered address of the applicant. -1 indicates missing value (Integer)

current_address_months_count: Months in currently registered address of the applicant. -1 indicates missing value (Integer)

customer_age: Applicant's age in years, rounded to the decade (Integer)

days_since_request: Number of days passed since application was done (Integer)

intended_balcon_amount: Initial transferred amount for application. Negative values indicate missing data (Float)

payment_type: Credit payment plan type (Categorical)

zip_count_4w: Number of applications within the same zip code in the last 4 weeks (Integer)

velocity_6h: Average number of applications per hour in the last 6 hours. Negative values may indicate anomalies or missing data (Float)

velocity_24h: Average number of applications per hour in the last 24 hours (Float)

velocity_4w: Average number of applications per hour in the last 4 weeks (Float)

bank_branch_count_8w: Number of total applications in the selected bank branch in the last 8 weeks (Integer)

date_of_birth_distinct_emails_4w: Number of emails for applicants with the same date of birth in the last 4 weeks (Integer)

employment_status: Employment status of the applicant (Categorical)

credit_risk_score: Internal score of application risk. Negative values indicate higher risk (Float)

email_is_free: Domain of application email (either free or paid) (Binary)

housing_status: Current residential status for the applicant (Categorical)

phone_home_valid: Validity of provided home phone (Binary)

phone_mobile_valid: Validity of provided mobile phone (Binary)

bank_months_count: Age of the applicant's previous bank account in months. -1 indicates missing value (Integer)

has_other_cards: Whether applicant has other cards from the same bank (Binary)

proposed_credit_limit: Applicant's proposed credit limit (Float)

foreign_request: Whether the origin country of the request is different from the bank's country (Binary)

source: Online source of application. Values include browser (INTERNET) or app (TELEAPP) (Categorical)

session_length_in_minutes: Length of the user session on the banking website. -1 indicates missing value (Integer)

device_os: Operating system of the device used for the request (e.g., Windows, macOS, Linux, X11, or other) (Categorical)

keep_alive_session: Whether the user opted to keep the session alive (Binary)

device_distinct_emails: Number of distinct emails associated with the device in the last 8 weeks. -1 indicates missing value (Integer)

device_fraud_count: Number of fraudulent applications submitted using the device (Binary)

month: Month when the application was made (Integer)

fraud_bool: Whether the application was fraudulent (1) or not (0) (Binary)

st_velocity_ratio: Ratio of velocity in last 6 hours to velocity in last 24 hours (Float) (Engineered Feature)

lt_velocity_ratio: Ratio of velocity in last 24 hours to velocity in last 4 weeks (Float) (Engineered Feature)

total_address_months: Total number of months spent at current and previous addresses combined (Integer) (Engineered Feature)

income_bin: Categorized income level – Low, Medium, High, Very High (Categorical) (Engineered Feature)

credit_risk_category: Categorized credit risk score – Low, Medium, High (Categorical) (Engineered Feature)

log_st_velocity_ratio: Log-transformed short-term velocity ratio (Float) (Engineered Feature)

log_lt_velocity_ratio: Log-transformed long-term velocity ratio (Float) (Engineered Feature)

7.2 Citations

LexisNexis. (2024, April 24). *Every dollar lost to a fraudster costs North America's financial institutions \$4.41 according to LexisNexis true cost of fraud study from LexisNexis Risk Solutions.* LexisNexis Risk Solutions. <https://risk.lexisnexis.com/global/en/about-us/press-room/press-release/20240424-tcof-financial-services-lending>

Lopez-Rojas, E. A., Elmir, A., & Axelsson, S. (2016, September). *PAYSIM: A financial mobile money simulator for fraud detection.* Research Gate. https://www.researchgate.net/publication/313138956_PAYSIM_A_FINANCIAL_MOBILE_MONEY_SIMULATOR_FOR_FRAUD_DETECTION